From MLPs to Trainable Models

VLC = Vision Learning Control

From MLPs to Trainable Models

Activations, objectives, and initialisation

Jonathon Hare and Antonia Marcu

Vision, Learning and Control
University of Southampton

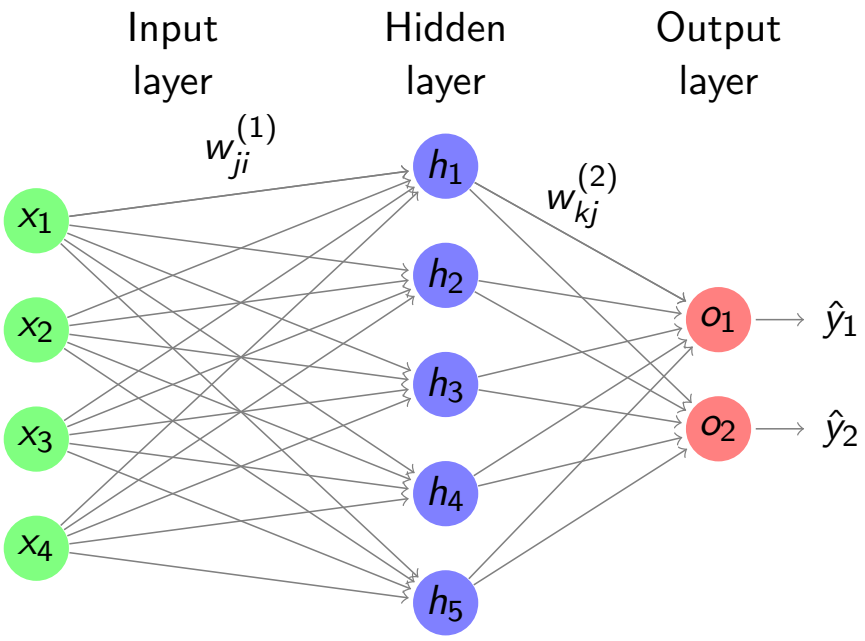
Topics

- Multilayer perceptrons as compositions of affine maps and nonlinearities
- Activations, logits, and loss functions
- The complete training pipeline
- Initialisation and the scale of signals and gradients
- Input scaling, weight initialisation, and internal normalisation

How Supervised Learning Typically Works

- Start by choosing a model-class: $\hat{y} = f(\boldsymbol{x}; \boldsymbol{W})$ where the model-class f is a way of using some numerical parameters, $\boldsymbol{W}$, to map each input vector $\boldsymbol{x}$ to a predicted output $\hat{y}$.
- *Learning* means adjusting the parameters to reduce the discrepancy between the true target output y on each training case and the output $\hat{y}$, predicted by the model.

Let's look at a Multilayer Perceptron (without biases)...



Without loss of generality, we can write the above as:

$$\hat{\boldsymbol{y}} = g(f(\boldsymbol{x}; \boldsymbol{W}^{(1)}); \boldsymbol{W}^{(2)}) = g(\boldsymbol{W}^{(2)} f(\boldsymbol{W}^{(1)} \boldsymbol{x}))$$

where f and g are activation functions.

Common Activation Functions

- Identity
- Sigmoid (aka Logistic)
- Hyperbolic Tangent (tanh)
- Rectified Linear Unit (ReLU) (aka Threshold Linear)

Final layer activations

$$\hat{\boldsymbol{y}} = g(\boldsymbol{W}^{(2)} f(\boldsymbol{W}^{(1)} \boldsymbol{x}))$$

- What form should the final layer function g take?
- It depends on the task (and on the chosen loss function)...
  - regression $\rightarrow$ typically linear
  - binary classification $\rightarrow$ typically Sigmoid (but...)
  - multilabel classification $\rightarrow$ typically Sigmoid (but...)
  - multiclass classification $\rightarrow$ typically Softmax (but...)

Softmax

$$\text{softmax}(\boldsymbol{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \qquad \forall i = 1, 2, \ldots, K$$

- The outputs are positive and sum to 1.
- The denominator contains every logit, so every output depends on every input.
- Let $\boldsymbol{s} = \text{softmax}(\boldsymbol{z})$. The entries of its Jacobian are:

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j).$$

Where δ_{ij} is the Kronecker delta: it is 1 if $i = j$, and 0 otherwise.

Softmax as a relaxation of one-hot argmax

- A multiclass decision selects the largest logit:

$$c = \arg\max_k z_k.$$

- The equivalent one-hot vector is discontinuous: a small change in the logits can switch the selected class.
- Softmax replaces the hard one-hot vector with a continuous, differentiable distribution:

$$\begin{aligned} \boldsymbol{z} &= [\ 1.1 \quad 4.0 \quad -0.1 \quad 2.3\ ] \\ \text{onehot}(\arg\max_k z_k) &= [\ 0 \quad 1 \quad 0 \quad 0\ ] \\ \text{softmax}(\boldsymbol{z}) &= [\ 0.044 \quad 0.797 \quad 0.013 \quad 0.146\ ] \end{aligned}$$

- The largest component stays associated with the predicted class, but the output retains information about the alternatives.

Temperature controls the relaxation

$$\text{softmax}(\boldsymbol{z}/T)_i = \frac{e^{z_i/T}}{\sum_{j=1}^{K} e^{z_j/T}}, \qquad T > 0.$$

- Lower temperatures sharpen the distribution:

$$\begin{aligned} T = 1.0 &: [\ 0.044 \quad 0.797 \quad 0.013 \quad 0.146\ ] \\ T = 0.6 &: [\ 0.008 \quad 0.937 \quad 0.001 \quad 0.055\ ] \\ T = 0.2 &: [\ 5.0\times10^{-7} \quad 0.9998 \quad 1.3\times10^{-9} \quad 0.0002\ ] \end{aligned}$$

- If the largest logit is unique, then as $T \to 0$:

$$\text{softmax}(\boldsymbol{z}/T) \longrightarrow \text{onehot}(\arg\max_k z_k).$$

- Higher temperatures flatten the distribution. As $T \to \infty$, it approaches the uniform distribution.

Ok, so let's talk loss functions

- The choice of loss function depends on:
  1. the task (e.g. classification/regression/something else)
  2. the activation function of the last layer
     - For numerical reasons
       the activation is often computed directly within the loss rather than being part of the model
- Some classification losses require *raw outputs* (e.g. a linear layer) of the network as their input
  - These are often called *unnormalised log probabilities* or *logits*
  - An example would be hinge-loss used to create a Support Vector Machine for binary classification
- There are many different loss functions we might encounter (MSE, Cross-Entropy, KL-Divergence, huber, L1 (MAE), CTC, Triplet, ...) for different tasks.

The Loss Function (measure of discrepancy)

Recall from Foundations of Machine Learning:

- Mean Squared Error (MSE) loss for a single data point is given by
  $\ell_{MSE}(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \sum_i (\hat{y}_i - y_i)^2 = (\hat{\boldsymbol{y}} - \boldsymbol{y})^\top (\hat{\boldsymbol{y}} - \boldsymbol{y})$
- We often multiply this by a constant factor of $\frac{1}{2}$ — can anyone guess/remember why?
- $\ell_{MSE}(\hat{\boldsymbol{y}}, \boldsymbol{y})$ is the predominant choice for regression problems with linear activation in the last layer
- For a classification problem with Softmax or Sigmoidal activations MSE can cause slow learning
  - Gradients of ℓ_{MSE} are proportional to the difference in target and predicted value, multiplied by the gradient of the activation function
  - **The Cross-Entropy loss function is generally a better choice in this case**

Binary Cross-Entropy

For the binary classification case:

$$\ell_{BCE}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- The cross-entropy loss function is non-negative, $\ell_{BCE} > 0$
- $\ell_{BCE} \approx 0$ when the prediction and targets are equal (i.e. $y = 0$ and $\hat{y} = 0$ or when $y = 1$ and $\hat{y} = 1$)
- With Sigmoidal final layer, $\frac{\partial \ell_{BCE}}{\partial \boldsymbol{w}_i^{(2)}}$ is proportional to just the error in the output $(\hat{y} - y)$ and therefore, the larger the error, the faster the network will learn!
- Note that the BCE is the negative log likelihood of the Bernoulli Distribution

Binary Cross-Entropy for multiple labels

In the case of multi-label classification with a network with multiple sigmoidal outputs you just sum the BCE over the outputs:

$$\ell_{BCE} = -\sum_{k=1}^{K} [y_k \log(\hat{y}_k) + (1 - y_k) \log(1 - \hat{y}_k)]$$

where K is the number of classes of the classification problem, $\hat{y} \in \mathbb{R}^K$.

Numerical Stability: The Log-Sum-Exp trick

$$\ell_{BCE}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- Consider what might happen early in training when the model might confidently predict a positive example as negative
  - $\hat{y} = \sigma(z) \approx 0 \implies z << 0$
  - if $\hat{y}$ is small enough, it will become 0 due to limited precision of floating-point representations
  - but then $\log(\hat{y}) = -\inf$, and everything will break!
- To tackle this problem implementations usually combine the sigmoid computation and BCE into a single loss function that you would apply to a network with linear outputs (e.g. `BCEWithLogitsLoss`).
- Internally, a trick called 'log-sum-exp' is used to *shift* the centre of an exponential sum so that only numerical underflow can potentially happen, rather than overflow

  .
  - Ultimately this means you'll always get a numerically reasonable result (and will avoid NaNs and Infs originating from this point).

Multiclass classification with Softmax Outputs

- We saw Softmax makes the output look like a probability distribution.
- Thus the obvious loss function is the *negative log likelihood* of the Categorical distribution (aka Multinoulli, Generalised Bernoulli, Multinomial with one sample)
- Log-Sum-Exp can be used for better numerical stability. PyTorch combines LogSoftmax with NLL in one loss and calls this "Categorical Cross-Entropy" (so you would use this with a **linear output layer**)

The complete training pipeline

1. Initialise the model parameters $\boldsymbol{\theta}$ and the optimiser state.
2. Draw a minibatch $(\boldsymbol{X}, \boldsymbol{y})$ and run the forward pass:

$$\hat{\boldsymbol{y}} = f(\boldsymbol{X}; \boldsymbol{\theta}), \qquad \mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} \ell(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i).$$

3. Use reverse-mode automatic differentiation to compute $\nabla_{\boldsymbol{\theta}} \mathcal{L}$.
4. Update the parameters and repeat with another minibatch.

One PyTorch training step

`optimizer.zero_grad()` → `loss = criterion(model(x), y)` → `loss.backward()` → `optimizer.step()`

Why zero initialisation fails

- Suppose every hidden unit in a layer starts with the same incoming weights ($\boldsymbol{w}$) and bias.
- The units compute the same activation h and receive the same gradient:

$$\boldsymbol{w}_j = \boldsymbol{w}_k \quad \Longrightarrow \quad h_j = h_k \quad \text{and} \quad \nabla_{\boldsymbol{w}_j} \mathcal{L} = \nabla_{\boldsymbol{w}_k} \mathcal{L}.$$

- An update preserves this equality. The units remain copies of one another, so the layer cannot learn distinct features.
- Random weight initialisation breaks this symmetry.
  - Biases may still be initialised to zero because the random weights have already broken the symmetry.

The scale of the initial weights matters

- Each layer transforms the scale of its inputs. Let $h^{(r)}$ denote an activation after layer r, and define its second moment as

$$q^{(r)} = \mathbb{E}[(h^{(r)})^2], \qquad q^{(r)} \approx c\, q^{(r-1)}.$$

- Across a network of depth D:

$$q^{(D)} \approx c^D q^{(0)}.$$

- If $c = 0.5$, then after 20 layers the signal is scaled by about 10^{-6}.
- If $c = 2$, the same network amplifies it by about 10^6.
- Backpropagation has the same repeated-multiplication problem. We want activations and gradients to stay at a useful scale.

Variance propagation through a linear layer

For one output of a linear layer,

$$z_j = \sum_{i=1}^{n_{\text{in}}} w_{ji} x_i.$$

- Assume the inputs and weights are independent, zero-centred, and identically distributed.
- If $\text{Var}[w_{ji}] = \sigma_w^2$ and $\text{Var}[x_i] = q$, then

$$\text{Var}[z_j] = n_{\text{in}}\, \sigma_w^2 q.$$

- Preserving the forward variance suggests $\sigma_w^2 \approx 1/n_{\text{in}}$.
- Applying the same argument to the backward pass suggests $\sigma_w^2 \approx 1/n_{\text{out}}$.

Xavier and Glorot initialisation

- When $n_{\text{in}} \neq n_{\text{out}}$, one weight variance cannot exactly preserve scale in both directions.
- Xavier initialisation uses the compromise

$$\text{Var}[w] = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

- Common forms are

$$w \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

or

$$w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right].$$

- This reasoning fits linear activations and tanh near its approximately linear region.

Why Xavier under-scales ReLU networks

Let $q^{(r)} = \mathbb{E}[(h^{(r)})^2]$. If the pre-activations z are symmetric around zero, then

$$\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2}\mathbb{E}[z^2].$$

- With equal fan-in and fan-out, Xavier gives $\text{Var}[w] \approx 1/n_{\text{in}}$.
- A ReLU layer therefore gives approximately

$$q^{(r)} \approx \frac{1}{2} n_{\text{in}} \text{Var}[w] q^{(r-1)} \approx \frac{1}{2} q^{(r-1)}.$$

- Repeated ReLU layers can shrink the activation scale by roughly 2^{-D}. The backward pass loses a similar factor because only the active ReLUs transmit gradients.

He and Kaiming initialisation

- He initialisation compensates for the factor of $\frac{1}{2}$ introduced by ReLU:

$$\mathrm{Var}[w] = \frac{2}{n_{\mathrm{in}}}.$$

- Now the activation second moment is approximately preserved:

$$q^{(r)} \approx \frac{1}{2} n_{\mathrm{in}} \frac{2}{n_{\mathrm{in}}} q^{(r-1)} = q^{(r-1)}.$$

- Common choices are

$$w \sim \mathcal{N}\left(0, \frac{2}{n_{\mathrm{in}}}\right) \quad \text{or} \quad w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\mathrm{in}}}}, \sqrt{\frac{6}{n_{\mathrm{in}}}}\right].$$

Bias initialisation and PyTorch defaults

- Hidden-layer biases can usually start at zero. Random weights already break the symmetry between units.
- An output bias can encode a useful prior. For binary classification with positive rate p:

$$b = \log \frac{p}{1-p}.$$

- `nn.Linear` currently draws its weights and biases uniformly from

$$\left[-\frac{1}{\sqrt{n_{\mathrm{in}}}}, \frac{1}{\sqrt{n_{\mathrm{in}}}}\right].$$

  This historical default is not the standard He initialisation for a ReLU layer.
- Initialise explicitly when the activation assumption matters:

```
nn.init.kaiming_normal_(layer.weight,
    nonlinearity="relu")
nn.init.zeros_(layer.bias)
```

PyTorch `nn.Linear` source

Input scaling, initialisation, and internal normalisation

Mechanism	What it controls	When it acts
Input scaling	The scale and conditioning of the data entering the first layer	Before training. Estimate scaling statistics from the training data only.
Weight initialisation	The starting scale and symmetry of the parameters	Once, when the model is constructed. Match it to the activation.
Internal normalisation	The scale of learned activations inside the network	During each forward pass. BatchNorm and LayerNorm use different statistics.

- These mechanisms work together. Normalisation does not make careful data scaling or sensible initialisation irrelevant.
- We will return to internal normalisation in later lectures.